

Decoupling Vision, Language, and Action for Efficient Multi-Task Robot Policies

Xiatao Sun^{1*}, Chen Liang¹, Ziyao Zeng¹, Qian Wang¹,
Haoyang Zhang^{2,3}, Yue Sun³, Qiucheng Li³, and Daniel Rakita¹

Abstract—Vision-Language-Action (VLA) models attach an action module to a Vision-Language Model (VLM) with billions of parameters and pay for that backbone at every control step. For a low-level manipulation policy, this cost may be unnecessary: the VLM supplies vision and language embeddings, and recent standalone vision encoders and encoder-only language models now match or exceed large VLMs on visual embedding and language understanding benchmarks. We study this question with a controlled experiment. Holding the demonstrations, the training budget, the tasks, and the measurement platform fixed, we vary the vision encoder, the language encoder, and the action head of a decoupled policy and compare against seven VLA baselines. The study yields the Decoupled Embodiment Model (DEM), which pairs a fine-tuned DINOv3 encoder and a frozen NeoBERT encoder with a MeanFlow head that generates each action chunk in a single forward pass. On 18 simulated manipulation tasks with held-out language paraphrases and randomized scenes, and on three real-robot tasks, DEM achieves observed success comparable to state-of-the-art VLM-backbone policies under our evaluation protocol, while running at eight to seventeen times their inference frequency and drawing six to fifteen times less energy per inference. Within this task scope, modern decoupled components offer a better success–latency–energy trade-off.

I. INTRODUCTION

Most Vision-Language-Action (VLA) models today are built the same way: take a large Vision-Language Model (VLM) and attach an action module [1, 2, 3]. The backbone costs billions of parameters, and the robot pays for it at every control step. $\pi_{0.5}$ [4] spends on the order of 100 ms per inference, so its perception-to-action loop runs at a few hertz. Action chunking [5] and asynchronous execution [6] hide the latency but do not shorten the loop underneath.

We ask whether this cost is necessary for a low-level manipulation policy. Inside a VLA, the VLM’s job is to encode images and an instruction into an embedding that conditions the action module. That job does not generate language, which is what the decoder-only VLM is built and sized for; autoregressive VLAs such as OpenVLA do run the decoder, but to emit action tokens, and most recent VLAs hand even that to a separate action module. Transformer language models split early into encoder-only models for understanding [7] and decoder-only models for generation [8]. A VLM may still contribute joint vision-language knowledge that separate encoders lack, so the question is empirical: how

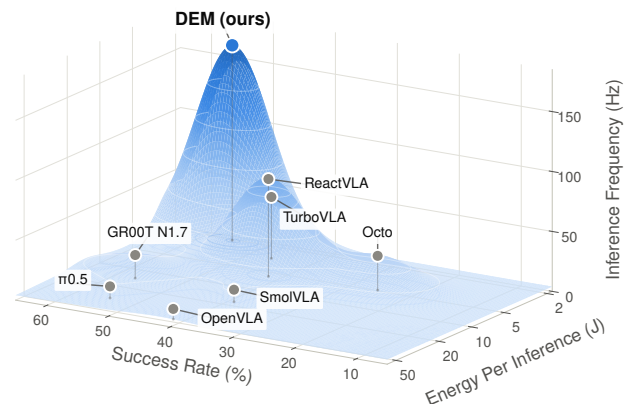


Fig. 1: **Success against inference cost.** Success rate on the 18 RoboCasa tasks versus inference frequency and energy per inference. DEM reaches success comparable to the VLM-backbone VLAs at a fraction of their cost.

much success does the backbone buy on the tasks a policy is trained for?

That choice made sense in 2023. Earlier generalist policies were already decoupled: RT-1 [9] ran EfficientNet alongside a frozen sentence encoder in 35M parameters, and Octo [10] conditioned a small transformer on frozen T5 embeddings. What limited them was the quality of their components: encoder-only language models had seen little new investment since ChatGPT pulled pretraining toward decoder-only models, and robotics, lacking native training data, followed the strongest available checkpoints into the VLM [2]. DINOv3 [11] and NeoBERT [12] have since restored frontier pretraining for standalone encoders, which now match the representations of decoder-only models several times their size (Section II).

A VLM backbone may be unnecessarily costly for low-level control policies. Such policies must be reactive and run under energy constraints, whereas LLMs and VLMs scale without regard to efficiency and live on remote servers that reactive control cannot call. On the robot, autoregressive VLAs run at 3–5 Hz against the 25–50 Hz that high-frequency control needs [13], and asynchronous execution hides the delay without shortening the reaction time [6].

Because inference speed and power both matter, we test the alternative directly: a decoupled policy with a compact vision encoder, an encoder-only language model, and an action

¹Department of Computer Science, Yale University, New Haven, CT, USA.

²Peking University, Beijing, China.

³Digients, Singapore.

*Corresponding author: xiatao.sun@yale.edu

head, with no VLM in the loop. We run a controlled study over the three modules, in which every variant is trained on the same demonstrations for the same number of steps and evaluated under the same protocol, so architecture is the only variable. The study yields the **Decoupled Embodiment Model (DEM)**. Thinking in decoupled terms also led us to re-examine the action module: an ablation over action heads (Section V-E) led us to MeanFlow [14], which produces a whole action chunk in one forward pass and accounts for a large share of DEM’s speed. On 18 RoboCasa tasks with held-out language paraphrases and randomized scenes, DEM achieves observed success comparable to VLAs seven to nine times its size under our evaluation protocol, while drawing six to fifteen times less energy per inference and running inference eight to seventeen times faster.

This paper makes three contributions.

- A controlled study of the vision encoder, the language encoder, and the action head of a decoupled policy on the same demonstrations, tasks, training budget, and measurement platform, against seven prior and concurrent VLAs, on 18 simulated tasks and three real-world tasks.
- Evidence that, within this task scope, modern decoupled components form a better success–latency–energy trade-off than VLM-backbone policies: comparable observed success at eight to seventeen times the inference frequency and six to fifteen times less energy per inference.
- An open release of our code¹ to support future research on efficient multi-task robot policies.

II. RELATED WORK

Decoupled policies and VLM backbones. The first language-conditioned generalist policies kept separate encoders: RT-1 [9] paired an EfficientNet with a frozen sentence encoder, Octo [10] conditioned a small transformer on frozen T5 embeddings, and BAKU [15] put interchangeable action heads behind separate encoders. Their components dated from 2019: encoder-only language modeling saw little follow-up after RoBERTa [16], and no pretrained visual representation of the time worked across embodied tasks [17]. As pretraining investment moved to decoder-only generation [8], robotics repurposed VLMs as embedding providers, and the recipe settled into OpenVLA [1], the π series [2, 4], and GR00T [3]. The backbone’s latency is handled by action chunking [5], parallel decoding [13], real-time chunk stitching [6], smaller or truncated backbones [18, 19], and token caching or early exit [20, 21], all of which keep the decoder-only structure. A separate line makes compact policies cheaper without a VLM, through low-rank or geometry-structured training of diffusion policies [22, 23], pose- or mesh-based observations [24, 25], learned viewpoint selection [26], and attention shaping against shortcut learning [27].

Encoder revival. The components that limited the decoupled policies have since improved. DINOv3 [11] features

exceed the language-supervised encoders that current VLMs use as vision towers on dense tasks and match them on classification. ModernBERT [16], NeoBERT [12], and mmBERT [28] renewed encoder pretraining, and paired-training experiments that hold data, architecture, and parameter count fixed show encoders beating decoders on representation tasks, with a 150M encoder above a 400M decoder on MNLI [29]. Inside VLMs, with the language model held fixed, perceptual performance is set by the vision tower [30]. Whether these encoders can replace the VLM inside a policy is the question this paper tests.

Concurrent work. TurboVLA [31] and ReactVLA [32] arrive at a similar design and evaluate it on LIBERO, where a 0.54M-parameter policy conditioned on a task index reaches 95.1% [33], so parity there says little about capability. MeanFlow [14] has entered manipulation through per-task point-cloud policies [34] and, concurrently with our work, as the action expert of language-conditioned policies [35, 32]. We evaluate on RoboCasa [36] with held-out paraphrases and randomized scenes, vary the vision encoder, the language encoder, and the action head under one protocol, and include TurboVLA and ReactVLA as baselines.

III. DEM: DECOUPLED EMBODIMENT MODEL

DEM does not share a backbone across modalities. A vision encoder, a language encoder, and an action head are three separate networks whose outputs meet only in the cross-attention inside the head (Fig. 2). The specific modules, DINOv3 ConvNeXt-B for vision, NeoBERT for language, and MeanFlow for the action head, were selected by the controlled study of Sections V-C to V-E.

A. Vision encoder

Each camera frame passes through a DINOv3 ConvNeXt-B [11]. We take the stage-3 feature map, a 16×16 grid of 512-dimensional features for a 256-pixel input, and flatten it into 256 tokens per camera; with a scene camera and a wrist camera the vision pathway emits $N_v = 512$ tokens per control step. The encoder is initialized from the public DINOv3 checkpoint and fine-tuned together with the head, which the ablation in Section V-C supports.

B. Language encoder

The instruction goes through NeoBERT [12], an encoder-only language model with 222M parameters as loaded. We pad or truncate the instruction to $N_l = 32$ tokens and keep the final-layer hidden state of every token with its attention mask rather than a pooled sentence vector. NeoBERT stays frozen, since fine-tuning it did not change success in our experiments (Section V-D). Because the language tokens are computed without reference to the images, the language pathway is optional at each step: when the instruction has not changed, the policy reuses the cached tokens, and when a new command arrives, it re-encodes. A VLM that fuses the instruction with the images inside one attention stack, as $\pi_{0.5}$ does, cannot skip this step without changing its attention layout, so it re-encodes the instruction at every step.

¹<https://github.com/Apollo-Lab-Yale/decoupled-embodiment-model>

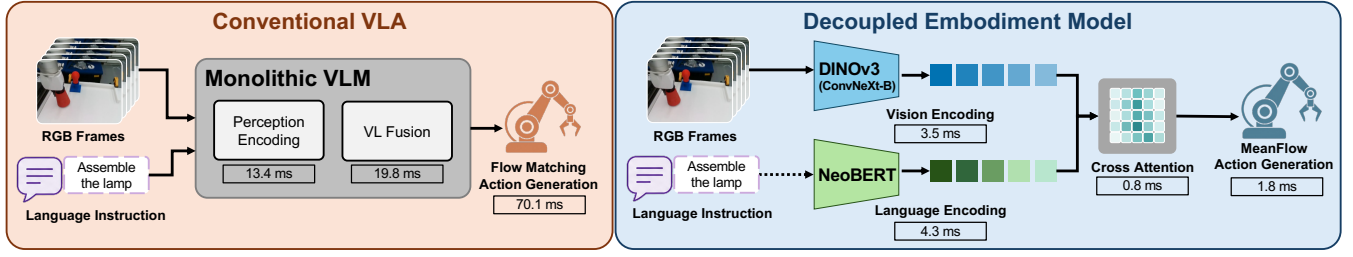


Fig. 2: **Conventional VLA versus DEM.** Left: a conventional VLA ($\pi_{0.5}$ [4]) routes images and instruction through one VLM and generates actions by multi-step flow matching; the instruction is re-encoded at every step. Right: DEM encodes images with DINOv3 ConvNeXt-B and the instruction with NeoBERT, fuses them by cross-attention inside the action head, and generates the chunk with MeanFlow in one forward pass. The dashed language pathway runs only when the instruction changes. Boxes give per-stage latencies and sum to the figures of Table II.

C. Action head

The head generates a chunk of H actions $\mathbf{a} \in \mathbb{R}^{H \times A}$, with A the action dimension, as a MeanFlow model [14]. Flow matching learns the instantaneous velocity $v(\mathbf{z}_t, t)$ of the interpolation $\mathbf{z}_t = (1 - t)\mathbf{a} + t\epsilon$, $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and recovers $\mathbf{a}$ from noise by integrating v over many small steps. MeanFlow instead learns the average velocity over an interval $[r, t]$,

$$\mathbf{u}(\mathbf{z}_t, r, t) = \frac{1}{t - r} \int_r^t v(\mathbf{z}_\tau, \tau) d\tau, \quad (1)$$

so that the displacement across the interval is a single product, $\mathbf{z}_r = \mathbf{z}_t - (t - r)\mathbf{u}(\mathbf{z}_t, r, t)$. Differentiating Eq. (1) with respect to t gives the MeanFlow identity

$$\mathbf{u}(\mathbf{z}_t, r, t) = v(\mathbf{z}_t, t) - (t - r) \frac{d}{dt} \mathbf{u}(\mathbf{z}_t, r, t), \quad (2)$$

which the network $\mathbf{u}_\theta$ is trained to satisfy. Along the interpolation $v = \epsilon - \mathbf{a}$ is known in closed form, and the total derivative $\frac{d}{dt} \mathbf{u}_\theta = v \partial_{\mathbf{z}} \mathbf{u}_\theta + \partial_t \mathbf{u}_\theta$ costs one Jacobian-vector product. The training loss is

$$\mathcal{L} = \mathbb{E} \left[w \left\| \mathbf{u}_\theta(\mathbf{z}_t, r, t \mid \mathbf{C}) - \text{sg}(v - (t - r) \frac{d}{dt} \mathbf{u}_\theta) \right\|_2^2 \right], \quad (3)$$

where sg stops gradients through the target, $\mathbf{C}$ is the conditioning context of Sec. III-D, and $w = (\|\cdot\|_2^2 + c)^{-p}$ is the adaptive weight of [14] with $c = 10^{-3}$ and $p = 0.5$, which that paper reports as competitive with its default $p = 1$. The pair (r, t) is drawn from a logit-normal distribution and set equal with probability 0.5, against 0.75 in the MeanFlow default, in which case Eq. (3) reduces to the flow-matching loss and anchors training.

At inference the head runs once. Starting from $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$,

$$\hat{\mathbf{a}} = \epsilon - \mathbf{u}_\theta(\epsilon, 0, 1 \mid \mathbf{C}), \quad (4)$$

which is the displacement identity with $r = 0$ and $t = 1$.

D. Assembling the modules

The three modules meet inside the head as one set of context tokens. Let $\mathbf{V} \in \mathbb{R}^{N_v \times d_v}$ be the vision tokens, $\mathbf{L} \in \mathbb{R}^{N_l \times d_l}$ the language tokens with mask $\mathbf{m} \in \{0, 1\}^{N_l}$, and

$\mathbf{q} \in \mathbb{R}^{d_p}$ the proprioceptive state. Each is projected to the head width d and tagged with a learned modality embedding,

$$\mathbf{C} = [\mathbf{V}\mathbf{W}_v + \mathbf{e}_v \parallel \mathbf{L}\mathbf{W}_l + \mathbf{e}_l \parallel \mathbf{q}^\top \mathbf{W}_p + \mathbf{e}_p], \quad (5)$$

where $\parallel$ concatenates along the token axis, so $\mathbf{C} \in \mathbb{R}^{(N_v + N_l + 1) \times d}$ holds 545 context tokens per step in our configuration. The noisy chunk enters as H action tokens, $\mathbf{h}^{(0)} = \mathbf{z}_t \mathbf{W}_a + \mathbf{P}$ with a learned position embedding $\mathbf{P} \in \mathbb{R}^{H \times d}$, and the interval (r, t) enters through a time embedding $\mathbf{c} = \phi_r(r) + \phi_t(t)$. Each of the D blocks applies self-attention among the action tokens and a feed-forward layer, both modulated by adaptive layer normalization from $\mathbf{c}$ as in DiT [37], and then reads the context:

$$\tilde{\mathbf{h}} = \text{AdaLNBlock}(\mathbf{h}^{(i)}, \mathbf{c}), \quad (6)$$

$$\mathbf{h}^{(i+1)} = \tilde{\mathbf{h}} + \mathbf{g}_i \odot \text{CrossAttn}(\text{LN}(\tilde{\mathbf{h}}), \mathbf{C}, \mathbf{m}), \quad (7)$$

where the action tokens are the queries, the context tokens are the keys and values, $\mathbf{m}$ is extended with ones over the vision and proprioceptive tokens so that only padded language tokens are masked, and $\mathbf{g}_i \in \mathbb{R}^d$ is a per-channel gate initialized at zero, so each block starts as an unconditional denoiser and learns how much of the scene and the instruction to read. A final layer normalization and a zero-initialized linear layer map $\mathbf{h}^{(D)}$ to $\mathbf{u}_\theta \in \mathbb{R}^{H \times A}$.

In our configuration $d = 768$, $D = 8$, and the head has 107M parameters. The head is the only module that consumes $\mathbf{C}$, so swapping either encoder changes nothing but $\mathbf{W}_v$ or $\mathbf{W}_l$, and every variant in Sections V-C to V-E plugs into the same head. The vision encoder is fine-tuned from its public checkpoint, the language encoder stays frozen, and the head trains from scratch.

IV. EXPERIMENTAL SETUP

Simulation setup. All simulated experiments run on 18 atomic tasks in the RoboCasa kitchens [36] (Fig. 3), the tasks for which the NVIDIA PhysicalAI-Robotics-Manipulation-Kitchen-Demos dataset [38] provides demonstrations; they span the eight skill categories of the RoboCasa paper and add tasks on appliances (toaster oven, blender, stand mixer, dishwasher, kettle) beyond the original atomic-task list. Training uses 500 episodes per task. The dataset ships one instruction

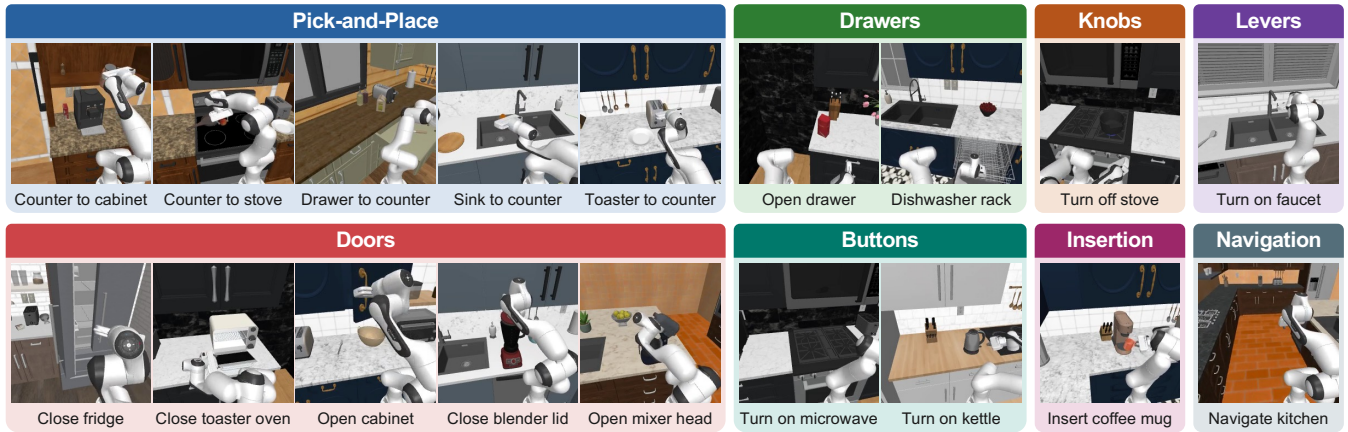


Fig. 3: **Simulated tasks.** One demonstration frame per task, grouped by the skill categories of Table I.

per task; to avoid training on a single string, we generate 50 paraphrases of each instruction with Claude Opus 4.8 [39] and sample one per training episode for every policy. At evaluation, every rollout uses one of 10 further paraphrases that never appear in training. Evaluation layouts and object placements are drawn from RoboCasa’s randomization and are not those of the demonstrations. Every policy and every variant is trained for 300k steps on the same demonstrations and evaluated over 100 rollouts per task. With 100 rollouts, the binomial standard error of a task’s success rate is at most 5 points and that of the 18-task mean about 1.2 points, so we treat differences of up to 2.5 points in the mean as within noise.

Baselines. We compare against OpenVLA [1], Octo [10], SmolVLA [19], $\pi_{0.5}$ [4], GR00T N1.7 [40], TurboVLA [31], and ReactVLA [32], all fine-tuned on the same demonstrations with their official implementations and default recipes, updating all parameters. DEM and the chunked baselines generate 16 actions per inference and replan every eight control steps.

Compute. Training runs on eight H100 GPUs. Evaluation and every latency and power measurement run on one workstation with an RTX PRO 6000 GPU, an AMD Ryzen Threadripper PRO 9965WX CPU, and 128 GB of memory. Latency covers the policy’s forward pass at batch size 1 and excludes camera capture, preprocessing, and robot communication, which are common to all policies; energy is the mean GPU power during inference times the median latency, with idle draw not subtracted.

Real-world setup. Real-world experiments use an xArm 7 mounted on a linear rail driven by a linear motor, which together form an eight-degree-of-freedom manipulation system. We evaluate three FurnitureBench [41] assembly tasks, drawer, lamp, and cabinet, with 200 teleoperated episodes per task, 100k training steps for every policy, the same paraphrase protocol, and 50 rollouts per task and method (standard error up to 7 points per task, about 4 points for the three-task mean). On the robot we compare against GR00T N1.7, the strongest VLM-backbone policy in simulation, $\pi_{0.5}$, the most widely deployed one and the architecture of

Fig. 2, and TurboVLA, the strongest concurrent decoupled policy; the remaining baselines score below TurboVLA in simulation while spending more energy per inference (Tables I and II).

V. RESULTS

A. Overall benchmark

Table I reports success rates on the 18 RoboCasa atomic tasks. DEM averages 55.6%, within 1.3 points of GR00T N1.7 (56.9%) and above $\pi_{0.5}$ (54.6%), differences inside the sampling error of Section IV, with 417M parameters, 195M of them active per step, against roughly 3B for either VLM-backbone policy (Table II). It has the highest success rate on drawers, levers, and buttons, trails GR00T N1.7 on pick-and-place, doors, and insertion, and has its largest deficit on the knob task (37.0% against 49.0%), which demands the finest positioning of the 18 tasks and is where the 16×16 token grid is most likely to limit precision. The two other decoupled policies sit below the VLM-backbone group, TurboVLA at 46.3% and ReactVLA at 41.9%, and Octo, which pairs 2024-era encoders with a diffusion head, is last at 25.6%. Octo and DEM share the decoupled structure and differ only in their components, which answers the question of Section II: with 2024 components the recipe finishes last, with 2025 components its observed success is comparable to the best VLM-backbone policy.

B. Inference frequency and energy

Table II reports the per-inference cost of each policy on the workstation of Section IV: median latency, the inference frequency it allows (its reciprocal), and the energy of one inference. DEM produces an action chunk in 6.1 ms, an inference rate of 163 Hz, and spends 2.07 J per inference. GR00T N1.7 and $\pi_{0.5}$, the two policies with comparable success rates, run at 19.3 and 9.7 Hz and spend 13.7 and 31.7 J. The two other decoupled policies are closer: TurboVLA runs at 51.2 Hz and 3.06 J and ReactVLA at 81.2 Hz and 5.82 J. With chunking, the inference rate bounds how often the policy can replan; reaction time adds the sensing and communication overhead shared by all policies.

TABLE I: **RoboCasa success rate (%)**. 18 tasks, 100 rollouts each, grouped by the eight RoboCasa skill categories [36] (task counts in parentheses); a category score is the mean over its tasks and Avg, the mean over all 18. The blender lid and stand-mixer head are counted with doors. All policies fine-tuned on the same demonstrations for 300k steps; standard error up to 5 points per task and about 1.2 points for Avg. Best in bold.

Method	Pick-place (5)	Doors (5)	Drawers (2)	Knobs (1)	Lever (1)	Buttons (2)	Insertion (1)	Navigation (1)	Avg. (18)
OpenVLA [1]	24.6	47.2	51.0	34.0	45.0	61.0	51.0	38.0	41.7
Octo [10]	13.0	32.8	30.0	25.0	35.0	28.5	20.0	35.0	25.6
SmolVLA [19]	27.8	43.4	44.0	25.0	48.0	54.0	44.0	37.0	39.2
$\pi_{0.5}$ [4]	34.2	55.8	72.5	44.0	60.0	78.0	63.0	64.0	54.6
GR00T N1.7 [40]	36.0	60.2	75.0	49.0	64.0	79.0	64.0	59.0	56.9
TurboVLA [31]	27.6	55.0	62.5	36.0	40.0	65.5	42.0	46.0	46.3
ReactVLA [32]	23.8	43.4	50.5	33.0	40.0	68.0	59.0	50.0	41.9
DEM (ours)	33.2	56.4	79.5	37.0	75.0	81.5	63.0	56.0	55.6

TABLE II: **Per-inference cost** on one RTX PRO 6000 at batch size 1: parameters, median latency, inference frequency (its reciprocal), and GPU energy (Section IV). DEM with the instruction cached and with NeoBERT run at every step. Best in bold. [†]ReactVLA as reproduced by us; its paper reports 390M for the action transformer alone. [‡]Total; 195M run at every step, NeoBERT (222M) only when the instruction changes.

Method	Params (M)	Latency (ms)	Freq. (Hz)	Energy (J)
OpenVLA	7541	118.0	8.5	45.13
Octo	202	35.0	28.6	4.88
SmolVLA	450	99.3	10.1	17.26
$\pi_{0.5}$	3600	103.3	9.7	31.68
GR00T N1.7	3000	51.8	19.3	13.71
TurboVLA	216	19.5	51.2	3.06
ReactVLA	2240 [†]	12.3	81.2	5.82
DEM, cached	417 [‡]	6.1	162.7	2.07
DEM, per-step	417 [‡]	10.4	96.0	3.09

The DEM figures use the on-demand language pathway of Fig. 2: the instruction is encoded when it arrives and its tokens are cached, so a control step pays for the vision encoder, the cross-attention, and one MeanFlow evaluation. A VLM that fuses the modalities in one attention stack has no such option without changing its attention layout (Section III-B). Running NeoBERT at every step instead raises DEM’s latency to 10.4ms (96Hz) and its energy to 3.09J per inference, which still leaves it the fastest policy in the table and within 0.03J of TurboVLA.

C. Vision encoder ablation

Table III swaps the vision encoder with the language encoder, fusion, and head fixed. Fine-tuning the encoder matters more than which encoder is chosen: the frozen DINOv3 ConvNeXt-B reaches 32.6%, the fine-tuned one 55.6%, and every fine-tuned encoder in the table lands between 51.8% and 55.6%. The two VLM rows use each VLM’s vision tower on its own with the same head and language encoder. Neither improves on DINOv3: they reach 53.9% and 51.8% with about five times the parameters, three times the latency, and four times the energy per inference.

These rows do not contradict Section III-B. Inside a VLM

TABLE III: **Vision encoder ablation**. Language encoder, fusion, and head fixed; encoders fine-tuned except where marked. The VLM rows use only the vision tower of PaliGemma 2 and Qwen3-VL 2B. Cost is per inference with language cached. Best in bold.

Encoder	Params (M)	Succ. (%)	Freq. (Hz)	Energy (J)
DINOv3 ViT-B/16	86.0	54.1	145.7	2.47
SigLIP-So400M	412.4	53.9	56.7	9.04
Qwen3-VL ViT	407.0	51.8	48.8	8.31
DINOv3 ConvNeXt-B, frozen	87.6	32.6	162.7	2.07
DINOv3 ConvNeXt-B, fine-tuned	87.6	55.6	162.7	2.07

the two towers share one attention stack, so the vision tower cannot run without the language model and the instruction is re-encoded at every step; in DEM each tower writes to a static context (Section III-D), so the language tokens can be cached. Taken apart, the VLM towers buy no success and cost three to four times more per inference; kept together, they also give up the caching.

D. Language encoder ablation

Table IV swaps the language encoder with the rest fixed. Two controls bound what language contributes. Without language tokens the policy reaches 11.3%: in a scene that affords several tasks it performs one of them at random. A one-hot code in place of the instruction raises this to 37.4%, still 18 points below NeoBERT. The text encoders resolve the task from held-out paraphrases, and the 2019-era encoders trail: T5-base at 44.5% and BERT-base at 47.8%, against 50.6% for mmBERT-base and 55.6% for NeoBERT. Fine-tuning NeoBERT does not change success within noise (55.1%), so DEM keeps it frozen. The two VLM language models, Gemma 2 from PaliGemma 2 and the Qwen3-VL text model, reach 55.0% and 54.3% with eight to twelve times NeoBERT’s parameters and two to three times its encoding latency, so the larger language model does not translate into success on these tasks. DEM pays the encoding cost in the table once per instruction, not per control step.

E. Action head and chunk length

Table V swaps the action head with the encoders and fusion fixed. The regression head of ACT runs as fast as

TABLE IV: **Language encoder ablation.** Vision encoder, fusion, and head fixed; encoders frozen except where marked. The VLM rows use only the language model of PaliGemma 2 and Qwen3-VL 2B; None removes the language tokens, One-hot replaces them with a one-hot code. Encode cost is for one instruction, paid once per instruction. Best in bold.

Encoder	Params (M)	Succ. (%)	Encode (ms)	Energy (J)
None	–	11.3	–	–
One-hot	–	37.4	–	–
T5-base	109.6	44.5	2.92	0.39
BERT-base	109.5	47.8	2.18	0.29
mmBERT-base	307	50.6	4.43	0.57
Gemma 2	2617	55.0	16.10	5.63
Qwen3-VL LM	1721	54.3	12.06	4.01
NeoBERT, fine-tuned	222	55.1	5.15	0.97
NeoBERT, frozen	222	55.6	5.15	0.97

TABLE V: **Action head ablation.** Encoders and fusion fixed; ACT and Diffusion Policy heads matched to MeanFlow in parameter count. Passes is head forward passes per chunk. Best in bold.

Action head	Passes	Succ. (%)	Freq. (Hz)	Energy (J)
Diffusion Policy	16	53.8	26.9	12.85
Flow matching	10	54.7	40.3	8.50
ACT	1	52.4	164.4	2.10
MeanFlow	1	55.6	162.7	2.07

MeanFlow, one pass per chunk at 164 Hz, and gives up 3.2 points of success. The two iterative heads keep the success, flow matching at 54.7% with ten passes, within noise of MeanFlow, and Diffusion Policy at 53.8% with sixteen, and pay for it in latency: 40 and 27 Hz against 163 Hz, and 8.5 and 12.9 J per chunk against 2.1 J. MeanFlow holds the success of the iterative heads, within noise, at the cost of the regression head; ACT’s deficit is at the edge of the sampling error.

Action chunking exists to amortize slow inference: a policy that needs 100 ms per call has to commit to many actions per call. Table VI sweeps the chunk length from 16 down to a single action, replanning every $H/2$ steps as Diffusion Policy does [42] and every step for $H = 1$; chunk length and replan interval change together, so the sweep does not separate the two. Success is flat between 16 and 8 (55.6% and 55.9%) and declines to 50.6% when DEM replans at every control step, which at 163 Hz is affordable and still exceeds every policy in Table I except the three at the top. Chunking is therefore not a required component of DEM, although on our tasks the longer chunks remain the better choice.

F. Real-world tasks

Table VII reports the real-world results on the tasks of Fig. 4, with 50 rollouts per task and method. GR00T N1.7 leads on all three tasks with an average of 68.0%; DEM follows two points behind on each task at 66.0%, a difference of three successes in 150 rollouts and inside the sampling error, ahead of $\pi_{0.5}$ (63.3%) and TurboVLA (53.3%). The

TABLE VI: **Chunk length.** H actions per inference, the first $H/2$ executed before replanning [42], every step for $H = 1$. Per-call latency is 6.14 ms for every H .

Chunk length H	16	8	4	2	1
Replan interval	8	4	2	1	1
Success (%)	55.6	55.9	52.7	51.4	50.6

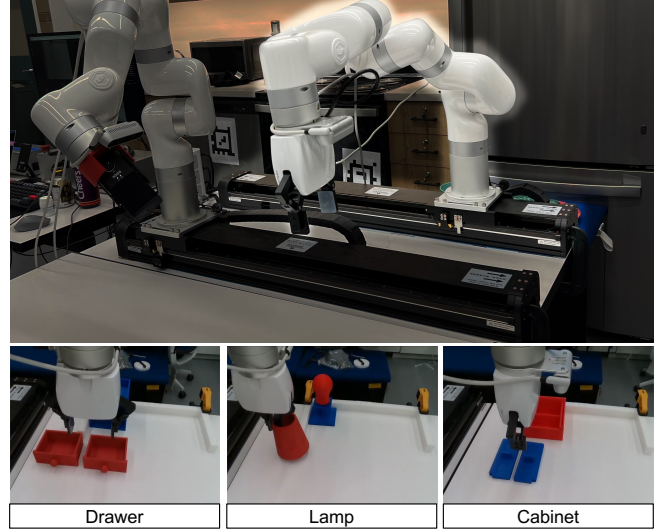


Fig. 4: **Real-world setup and tasks.** Top: an xArm 7 on a linear rail (highlighted), eight degrees of freedom; the second arm only carries the scene camera. Bottom: drawer, lamp, and cabinet assembly adapted from FurnitureBench [41].

ordering matches the simulation benchmark, where the same three policies finish within 2.3 points of one another. The architectures, inference stack, and workstation are the same as in simulation, so the costs of Table II apply unchanged: DEM reaches this success at 163 Hz and 2.07 J per inference, GR00T N1.7 at 19.3 Hz and 13.7 J.

VI. DISCUSSION

Within the scope of our tasks, what the policy needs from its backbone is an embedding, and modern standalone encoders supply it: DEM achieves observed success comparable to VLM-backbone policies under our protocol at a fraction of their latency and energy. Comparable is a descriptive statement; with 100 rollouts per task the study resolves differences of a few points and does not establish equivalence. The ablations locate where the cost of a VLM goes: its towers, used on their own inside DEM, add three to four times the latency and energy of DINOv3 and NeoBERT and no success (Tables III and IV), and the iterative heads that VLAs inherit from diffusion and flow matching cost four to six times the latency of MeanFlow for the same success (Table V). Two asymmetries shape the final design. Fine-tuning the vision encoder is worth 23 points, while fine-tuning the language encoder changes nothing: the workspace, the gripper, and the objects are specific to the robot and absent from web pretraining, whereas the instructions are

TABLE VII: **Real-world success rate (%)**. Three tasks of Fig. 4, 50 rollouts per task and method. Best in bold.

Method	Drawer	Lamp	Cabinet	Avg.
$\pi_{0.5}$	70	80	40	63.3
GR00T N1.7	74	84	46	68.0
TurboVLA	60	72	28	53.3
DEM	72	82	44	66.0

short and their vocabulary is ordinary English. Chunk length, in turn, trades five points of success against nothing in per-call latency (Table VI), so at 163 Hz chunking is a choice rather than a requirement imposed by inference time.

Several limits bound these conclusions. The evaluation tasks are the training tasks: scenes and instructions vary, but no rollout asks for an object or a task absent from training, so the experiments do not test the joint vision-language knowledge that a VLM’s pretraining might provide on unseen objects, instructions, or tasks; our results say only that this knowledge did not show up as success on the tasks studied here. The results show that the decoupled policy is cheaper at comparable success; whether its higher inference rate also buys success on tasks that demand fast reaction is left to future work.

VII. CONCLUSION

We studied, under one protocol, how much of a VLA’s success a decoupled policy reaches on multi-task manipulation. The configuration the study arrives at, DEM, pairs a fine-tuned DINOv3 encoder and a frozen NeoBERT encoder with a one-step MeanFlow head and achieves observed success comparable to VLM-backbone policies on 18 simulated tasks with held-out paraphrases and randomized scenes and on three real-world tasks, at a fraction of their latency and energy. On these tasks the VLM’s towers bring no additional success when used as parts, fine-tuning pays on the vision side and not on the language side, and one-step generation makes action chunking optional. The properties that make DEM efficient, a 163 Hz inference rate and an instruction encoded only when it changes, also suggest two directions for future work. Agentic robot systems, in which a language model plans and calls lower-level skills, are drawing growing interest, and a policy with these properties is a natural skill for such a system to call. Inference speed also matters for reinforcement learning: online RL of a billion-parameter VLA is limited by the compute its forward passes and parameter updates demand [43], so recent work runs it in large-scale parallel simulation [44], freezes the VLA and trains a small latent policy around it [45], or replaces the online loop with iterated offline updates [46]. A policy whose forward pass takes 6.1 ms lowers the cost of every step of that loop and may make direct online RL of a multi-task policy on the robot practical again.

ACKNOWLEDGMENT

The instruction paraphrases used for training and evaluation (Section IV) were generated with Claude Opus 4.8 [39].

REFERENCES

- [1] M. J. Kim *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Conference on Robot Learning (CoRL)*, 2024.
- [2] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Robotics: Science and Systems (RSS)*, 2025.
- [3] J. Bjorck *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [4] Physical Intelligence *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [5] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems (RSS)*, 2023.
- [6] K. Black, M. Y. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019.
- [8] T. B. Brown *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [9] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Robotics: Science and Systems (RSS)*, 2023.
- [10] Octo Model Team *et al.*, “Octo: An open-source generalist robot policy,” in *Robotics: Science and Systems (RSS)*, 2024.
- [11] O. Siméoni *et al.*, “DINOv3,” *arXiv preprint arXiv:2508.10104*, 2025.
- [12] L. Le Breton, Q. Fournier, M. El Mezouar, J. X. Morris, and S. Chandar, “NeoBERT: A next-generation BERT,” *arXiv preprint arXiv:2502.19587*, 2025.
- [13] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Robotics: Science and Systems (RSS)*, 2025.
- [14] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He, “Mean flows for one-step generative modeling,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [15] S. Haldar, Z. Peng, and L. Pinto, “BAKU: An efficient transformer for multi-task policy learning,” *arXiv preprint arXiv:2406.07539*, 2024.
- [16] B. Warner *et al.*, “Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference,” *arXiv preprint arXiv:2412.13663*, 2024.
- [17] A. Majumdar *et al.*, “Where are we in the search for an artificial visual cortex for embodied intelligence?” in *Advances in Neural Information Processing Systems*

(NeurIPS), 2023.

- [18] J. Wen *et al.*, “TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation,” *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3988–3995, 2025.
- [19] M. Shukor *et al.*, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [20] S. Xu, Y. Wang, C. Xia, D. Zhu, T. Huang, and C. Xu, “VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching,” *arXiv preprint arXiv:2502.02175*, 2025.
- [21] Y. Yue *et al.*, “DeeR-VLA: Dynamic inference of multimodal large language models for efficient robot execution,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [22] X. Sun, S. Yang, Y. Chen, F. Fan, Y. Liang, and D. Rakita, “Dynamic rank adjustment in diffusion policies for efficient and flexible training,” in *Robotics: Science and Systems (RSS)*, 2025.
- [23] X. Sun, Y. Wang, S. Yang, Y. Chen, and D. Rakita, “Hybrid diffusion policies with projective geometric algebra for efficient robot manipulation learning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [24] X. Sun, Y. Chen, and D. Rakita, “PRISM-DP: Spatial pose-based observations for diffusion-policies via segmentation, mesh generation, and pose tracking,” *arXiv preprint arXiv:2504.20359*, 2025.
- [25] Q. Wang, O. Abdellall, T. Gao, X. Sun, and D. Rakita, “Subsecond 3D mesh generation for robot manipulation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [26] X. Sun, F. Fan, Y. Chen, and D. Rakita, “Optimizing active perception for learning simultaneous viewpoint selection and manipulation with diffusion policy,” *arXiv preprint arXiv:2409.14615*, 2024.
- [27] X. Sun *et al.*, “Artificial foveated perception for mitigating shortcut learning in robotic foundation models,” *arXiv preprint arXiv:2607.10655*, 2026.
- [28] M. Marone, O. Weller, W. Fleshman, E. Yang, D. Lawrie, and B. Van Durme, “mmBERT: A modern multilingual encoder with annealed language learning,” *arXiv preprint arXiv:2509.06888*, 2025.
- [29] O. Weller, K. Ricci, M. Marone, A. Chaffin, D. Lawrie, and B. Van Durme, “Seq vs seq: An open suite of paired encoders and decoders,” in *International Conference on Learning Representations (ICLR)*, 2026.
- [30] S. Tong *et al.*, “Cambrian-1: A fully open, vision-centric exploration of multimodal LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [31] H. Xie *et al.*, “TurboVLA: Real-time vision-language-action model at 32 Hz on an RTX 4090 with <1 GB VRAM,” *arXiv preprint arXiv:2607.27205*, 2026.
- [32] Y. Guo, W. Chen, and J. Zhang, “ReactVLA: Fast and lightweight reactive robot manipulation via improved mean flow action generation,” *arXiv preprint arXiv:2606.14255*, 2026.
- [33] K. Sendai, T. Matsushima, and Y. Iwasawa, “MIN-ERVA: How small can a manipulation policy be and still solve LIBERO?” *arXiv preprint arXiv:2609.03715*, 2026.
- [34] J. Sheng, Z. Wang, P. Li, and M. Liu, “MP1: MeanFlow tames policy learning in 1-step for robotic manipulation,” in *AAAI Conference on Artificial Intelligence (AAAI)*, 2026.
- [35] Y. Chen, X. Ma, and B. Zhao, “Mean-flow based one-step vision-language-action,” *arXiv preprint arXiv:2603.01469*, 2026.
- [36] S. Nasiriany *et al.*, “RoboCasa: Large-scale simulation of everyday tasks for generalist robots,” in *Robotics: Science and Systems (RSS)*, 2024.
- [37] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [38] NVIDIA, “PhysicalAI-Robotics-Manipulation-Kitchen-Demos,” <https://huggingface.co/datasets/nvidia/PhysicalAI-Robotics-Manipulation-Kitchen-Demos>, 2025, Hugging Face dataset, CC-BY-4.0, accessed 2026-09-14.
- [39] Anthropic, “Claude Opus 4.8,” <https://www.anthropic.com/claude>, 2026, Large language model, accessed 2026-09-15.
- [40] NVIDIA, “GR00T-N1.7-3B model card,” <https://huggingface.co/nvidia/GR00T-N1.7-3B>, 2026, Hugging Face model card, first published 2026-02-25, accessed 2026-09-14.
- [41] M. Heo, Y. Lee, D. Lee, and J. J. Lim, “FurnitureBench: Reproducible real-world benchmark for long-horizon complex manipulation,” in *Robotics: Science and Systems (RSS)*, 2023.
- [42] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.
- [43] Y. Guo *et al.*, “Improving vision-language-action model with online reinforcement learning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [44] H. Zang *et al.*, “RLinf-VLA: A unified and efficient framework for reinforcement learning of vision-language-action models,” in *Robotics: Science and Systems (RSS)*, 2026, arXiv:2510.06710.
- [45] A. Wagenmaker *et al.*, “Steering your diffusion policy with latent space reinforcement learning,” in *Conference on Robot Learning (CoRL)*, 2025, arXiv:2506.15799.
- [46] Physical Intelligence *et al.*, “ $\pi_{0.6}^*$: a VLA that learns from experience,” *arXiv preprint arXiv:2511.14759*, 2025.